SQL-99: Schema Definition, Constraints, & Queries and Views

Chapter Outline

- Data Definition, Constraints, and Schema Changes
  - CREATE, DROP, and ALTER the descriptions of the tables (relations) of a database
  - Referential Integrity Options.
- Specifying Updates in SQL
  - **INSERT**, **DELETE**, and **UPDATE**
- Retrieval Queries in SQL
  - **SELECT** <attribute list>
    FROM <table list>
    [**WHERE** <condition>]
    [**GROUP BY** <grouping attribute(s)>]
    [**HAVING** <group condition>]
    [**ORDER BY** <attribute list>]
- Assertions, Triggers & Views

CREATE TABLE

- Specifies a new base relation by giving it:
  - a name &
  - specifying each of its attributes &
  - their data types (*INTEGER, FLOAT, DECIMAL(i,j), CHAR(n), VARCHAR(n)*)

- A constraint NOT NULL may be specified on an attribute

```
CREATE TABLE DEPARTMENT (
    DNAME           VARCHAR(10)     NOT NULL,
    DNUMBER         INTEGER         NOT NULL,
    MGRSSN          CHAR(9),
    MGRSTARTDATE    CHAR(9)   );
```

Naming tables

- Start with Alphabetical
- Special characters as (_, #,) and numbers are allowed
- Not case sensitive (emp, Emp, EMP)
- Unique name within account
- Not an sql reserved word as Select, Update, ….
- Not more that 30 characters

CREATE TABLE

- In SQL2, can use the CREATE TABLE command for specifying:
  - Primary Key attributes,
  - Secondary Keys, &
  - Referential Integrity Constraints (Foreign Keys).

```
CREATE TABLE DEPT (
  DNAME              VARCHAR(10)    NOT NULL,
  DNUMBER            INTEGER        NOT NULL,
  MGRSSN             CHAR(9),
  MGRSTARTDATE       CHAR(9),
  PRIMARY KEY (DNUMBER),
  UNIQUE (DNAME),
  FOREIGN KEY (MGRSSN) REFERENCES EMP  );
```

DROP TABLE

- Used to remove a relation (base table) and its definition
- The relation can no longer be used in queries, updates, or any other commands since its description no longer exists
- Example:

```
DROP TABLE  DEPENDENT;
```

ALTER TABLE

- Used to add an attribute to one of the base relations
  - The new attribute will have NULLs in all the tuples of the relation right after the command is executed; hence, the NOT NULL constraint is not allowed for such an attribute
- Example:

```
ALTER TABLE EMPLOYEE ADD JOB VARCHAR(12);
```

- The database users must still enter a value for the new attribute JOB for each EMPLOYEE tuple.
  - This can be done using the UPDATE command.

REFERENTIAL INTEGRITY OPTIONS

- We can specify **RESTRICT, CASCADE, SET NULL or SET DEFAULT** on referential integrity constraints (foreign keys)

```
CREATE TABLE DEPT (
   DNAME            VARCHAR(10)    NOT NULL,
   DNUMBER          INTEGER        NOT NULL,
   MGRSSN           CHAR(9),
   MGRSTARTDATE     CHAR(9),
   PRIMARY KEY (DNUMBER),
   UNIQUE (DNAME),
   FOREIGN KEY (MGRSSN) REFERENCES EMP
    ON DELETE SET DEFAULT
    ON UPDATE CASCADE);
```

REFERENTIAL INTEGRITY OPTIONS (continued)

```
CREATE TABLE EMP(
  ENAME       VARCHAR(30)    NOT NULL,
  ESSN        CHAR(9),
  BDATE       DATE,
  DNO         INTEGER  DEFAULT 1,
  SUPERSSN    CHAR(9),
  PRIMARY KEY (ESSN),
  FOREIGN KEY (DNO) REFERENCES DEPT
    ON DELETE SET DEFAULT
    ON UPDATE CASCADE,
  FOREIGN KEY (SUPERSSN) REFERENCES EMP
    ON DELETE SET NULL
    ON UPDATE CASCADE);
```

Additional Data Types in SQL2 and SQL-99

Has DATE, TIME, and TIMESTAMP data types

- **DATE:**
  - Made up of year-month-day in the format yyyy-mm-dd
- **TIME:**
  - Made up of hour:minute:second in the format hh:mm:ss
- **TIME(i):**
  - Made up of hour:minute:second plus i additional digits specifying fractions of a second
  - format is hh:mm:ss:ii...i

Additional Data Types in SQL2 and SQL-99 (contd.)

- **TIMESTAMP:**
  - Has both DATE and TIME components
- **INTERVAL:**
  - Specifies a relative value rather than an absolute value
  - Can be DAY/TIME intervals or YEAR/MONTH intervals
  - Can be positive or negative when added to or subtracted from an absolute value, the result is an absolute value

Specifying Updates in SQL

- There are three SQL commands to modify the database:
  - **INSERT**
  - **DELETE**
  - **UPDATE**

INSERT

- In its simplest form, it is used to add one or more tuples to a relation

- Attribute values should be listed in the *same order* as the attributes were specified in the **CREATE TABLE** command

INSERT (contd.)

- Example:

```
U1: INSERT INTO    EMPLOYEE
    VALUES         ('Richard','K','Marini', '653298653', '30-
                   DEC-52','98 Oak Forest,Katy,TX', 'M',
                   37000,'987654321', 4 )
```

- An alternate form of INSERT specifies explicitly the attribute names that correspond to the values in the new tuple
  - Attributes with NULL values can be left out
- Example: Insert a tuple for a new EMPLOYEE for whom we only know the FNAME, LNAME, and SSN attributes.

```
U1A: INSERT INTO   EMPLOYEE (FNAME, LNAME, SSN, DNO)
     VALUES        ('Richard', 'Marini', '653298653', 4)
```

INSERT (contd.)

- Important Note:
  - Only the constraints specified in the DDL commands are automatically enforced by the DBMS when updates are applied to the database

- Another variation of INSERT allows insertion of *multiple tuples* resulting from a query into a relation

DELETE

- Removes tuples from a relation
  - Includes a WHERE-clause to select the tuples to be deleted
  - Referential integrity should be enforced
  - Tuples are deleted from only *one table* at a time (unless CASCADE is specified on a referential integrity constraint)
  - A missing WHERE-clause specifies that *all tuples* in the relation are to be deleted; the table then becomes an empty table
  - The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause

DELETE (contd.)

- Examples:

```
U4A:  DELETE FROM   EMPLOYEE
      WHERE         LNAME='Brown'
```

```
U4B:  DELETE FROM   EMPLOYEE
      WHERE         SSN='123456789'
```

```
U4C:  DELETE FROM   EMPLOYEE
      WHERE         DNO  IN
                    ( SELECT  DNUMBER
                      FROM    DEPARTMENT
                      WHERE   DNAME='Research')
```

```
U4D:  DELETE FROM   EMPLOYEE
```

UPDATE

- Used to modify attribute values of one or more selected tuples
- A WHERE-clause selects the tuples to be modified
- An additional SET-clause specifies the attributes to be modified and their new values
- Each command modifies tuples *in the same relation*
- Referential integrity should be enforced

UPDATE (contd.)

- **Example**: Change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively.

```
U5:     UPDATE    PROJECT
        SET       PLOCATION = 'Bellaire',
                  DNUM = 5
        WHERE     PNUMBER=10
```

Retrieval Queries in SQL

- SQL has one basic statement for retrieving information from a database; the **SELECT** statement
- Important distinction between SQL and the formal relational model:
  - SQL allows a table (relation) to have two or more tuples that are identical in all their attribute values
  - Hence, an SQL relation (table) is a **multi-set** (sometimes called a **bag**) of tuples; it is *not* a set of tuples
- SQL relations can be constrained to be sets by specifying PRIMARY KEY or UNIQUE attributes, or by using the DISTINCT option in a query

Retrieval Queries in SQL (contd.)

- A **bag** or **multi-set** is like a set, but an element may appear more than once.
  - Example: {A, B, C, A} is a bag. {A, B, C} is also a bag that also is a set.
  - Bags also resemble lists, but the order is irrelevant in a bag.
- Example:
  - {A, B, A} = {B, A, A} as bags
  - However, [A, B, A] is not equal to [B, A, A] as lists

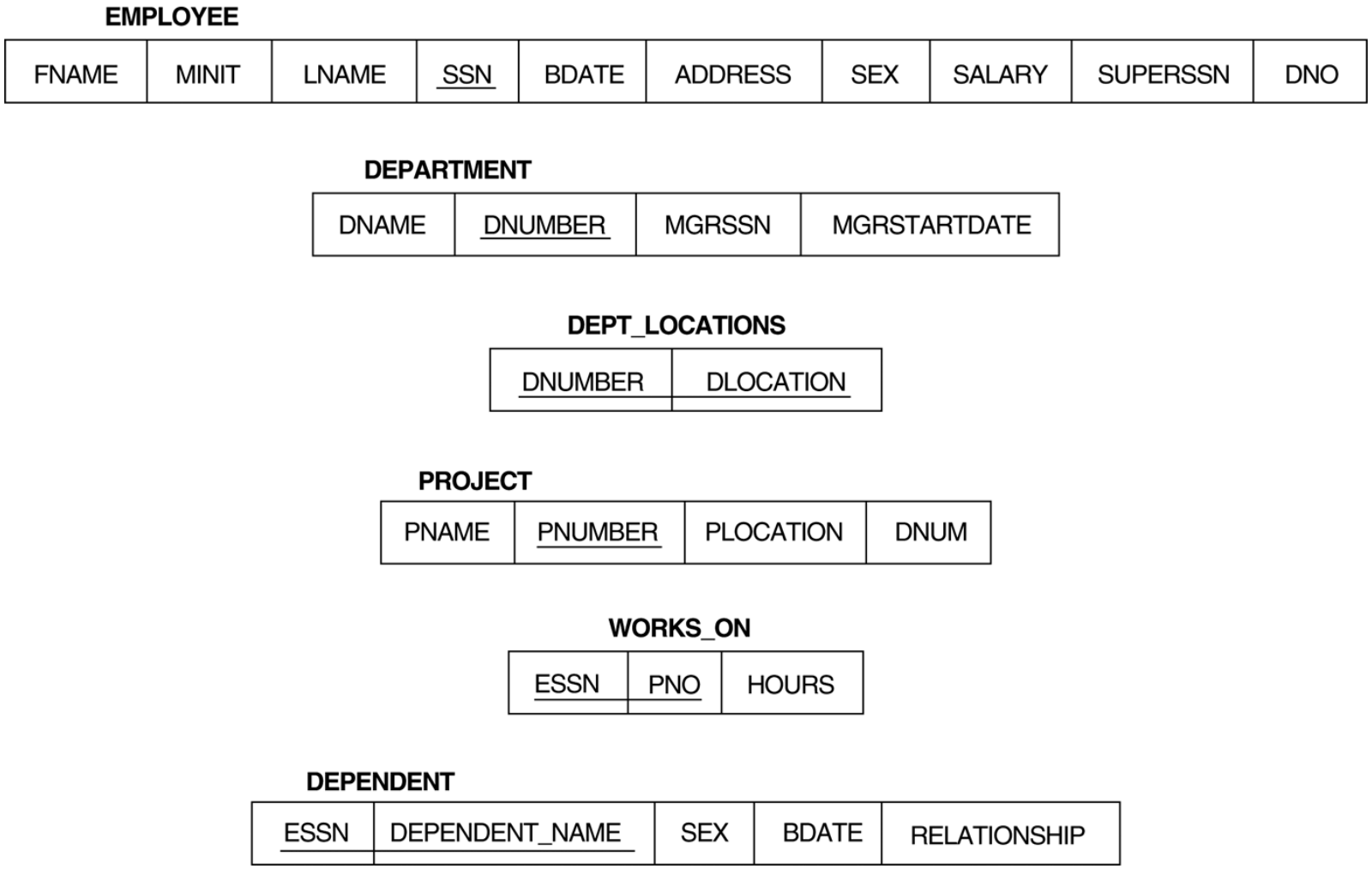
Retrieval Queries in SQL (contd.)

- Basic form of the SQL SELECT statement is called a *mapping* or a SELECT-FROM-WHERE *block*

```
SELECT   <attribute list>
  FROM   <table list>
 WHERE   <condition>
```

  - <attribute list> is a list of attribute names whose values are to be retrieved by the query
  - <table list> is a list of the relation names required to process the query
  - <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query

Relational Database Schema--Figure 5.5



EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
	1	Houston
	4	Stafford
	5	Bellaire
	5	Sugarland
	5	Houston

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

Simple SQL Queries

- Basic SQL queries correspond to using the following operations of the relational algebra:
  - SELECT
  - PROJECT
  - JOIN
- All subsequent examples use the COMPANY database

Simple SQL Queries (contd.)

- Example of a simple query on one relation

- **Query 0:** Retrieve the birthdate and address of the employee whose name is 'John B. Smith'.

```
Q0: SELECT   BDATE, ADDRESS
    FROM     EMPLOYEE
    WHERE    FNAME= 'John' AND MINIT='B'
             AND LNAME= 'Smith'
```

  - Note:
    The result of the query may contain duplicate tuples

Simple SQL Queries (contd.)

- **Query 1**: Retrieve the name and address of all employees who work for the 'Research' department.

```
Q1: SELECT    FNAME, LNAME, ADDRESS
    FROM      EMPLOYEE, DEPARTMENT
    WHERE     DNAME='Research' AND DNUMBER=DNO
```

  - (DNAME='Research') is a selection condition
    - (corresponds to a SELECT operation in relational algebra)
  - (DNUMBER=DNO) is a join condition
    - (corresponds to a JOIN operation in relational algebra)

Simple SQL Queries (contd.)

- **Query 2**: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birthdate.

```
Q2: SELECT   PNUMBER, DNUM, LNAME, BDATE, ADDRESS
    FROM     PROJECT, DEPARTMENT, EMPLOYEE
    WHERE    DNUM=DNUMBER AND MGRSSN=SSN
             AND PLOCATION='Stafford'
```

- In Q2, there are two join conditions
  - The join condition DNUM=DNUMBER relates a project to its controlling department
  - The join condition MGRSSN=SSN relates the controlling department to the employee who manages that department

Aliases, * and DISTINCT, Empty WHERE-clause

- In SQL, we can use the same name for two (or more) attributes as long as the attributes are in *different relations*
- A query that refers to two or more attributes with the same name must *qualify* the attribute name with the relation name by *prefixing* the relation name to the attribute name
- Example:

- **EMPLOYEE.**LNAME, **DEPARTMENT.**DNAME

ALIASES

- Some queries need to refer to the same relation twice
  - In this case, *ALIASES* are given to the relation name

- **Query 8:** For each employee, retrieve the employee's name, and the name of his or her immediate supervisor.

```
Q8:  SELECT    E.FNAME, E.LNAME, S.FNAME, S.LNAME
     FROM      EMPLOYEE E, EMPLOYEE S
     WHERE     E.SUPERSSN=S.SSN
```

  - In Q8, the alternate relation names E and S are called *aliases* or *tuple variables* for the EMPLOYEE relation
  - We can think of E and S as two different *copies* of EMPLOYEE; E represents employees in role of *supervisees* and S represents employees in role of *supervisors*

ALIASES (contd.)

- Aliasing can also be used in any SQL query for convenience
- Can also use the **AS** keyword to specify aliases

```
Q8:   SELECT   E.FNAME, E.LNAME,
               S.FNAME, S.LNAME
      FROM     EMPLOYEE AS E,
               EMPLOYEE AS S
      WHERE    E.SUPERSSN=S.SSN
```

UNSPECIFIED WHERE-clause

- A *missing WHERE-clause* indicates no condition; hence, all tuples of the relations in the FROM-clause are selected
  - This is equivalent to the condition WHERE TRUE
- Query 9: Retrieve the SSN values for all employees.

```
Q9:   SELECT   SSN
      FROM     EMPLOYEE
```

- Note:
  - If more than one relation is specified in the FROM-clause *and* there is no join condition, then the *CARTESIAN PRODUCT* of tuples is selected

UNSPECIFIED WHERE-clause (contd.)

- Example:

```
Q10:    SELECT    SSN, DNAME
        FROM      EMPLOYEE, DEPARTMENT
```

- Note:
  - It is extremely important not to overlook specifying any selection and join conditions in the WHERE-clause; otherwise, incorrect and very large relations may result

USE OF *

- To retrieve all the attribute values of the selected tuples, a * is used, which stands for *all the attributes*
- Examples:

```
Q1C:    SELECT    *
        FROM      EMPLOYEE
        WHERE     DNO=5

Q1D:    SELECT    *
        FROM      EMPLOYEE, DEPARTMENT
        WHERE     DNAME='Research' AND
                  DNO=DNUMBER
```

USE OF DISTINCT

- SQL does not treat a relation as a set; duplicate tuples can appear
- To eliminate duplicate tuples in a query result, the keyword **DISTINCT** is used
- For example, the result of Q11 may have duplicate SALARY values whereas Q11A does not have any duplicate values

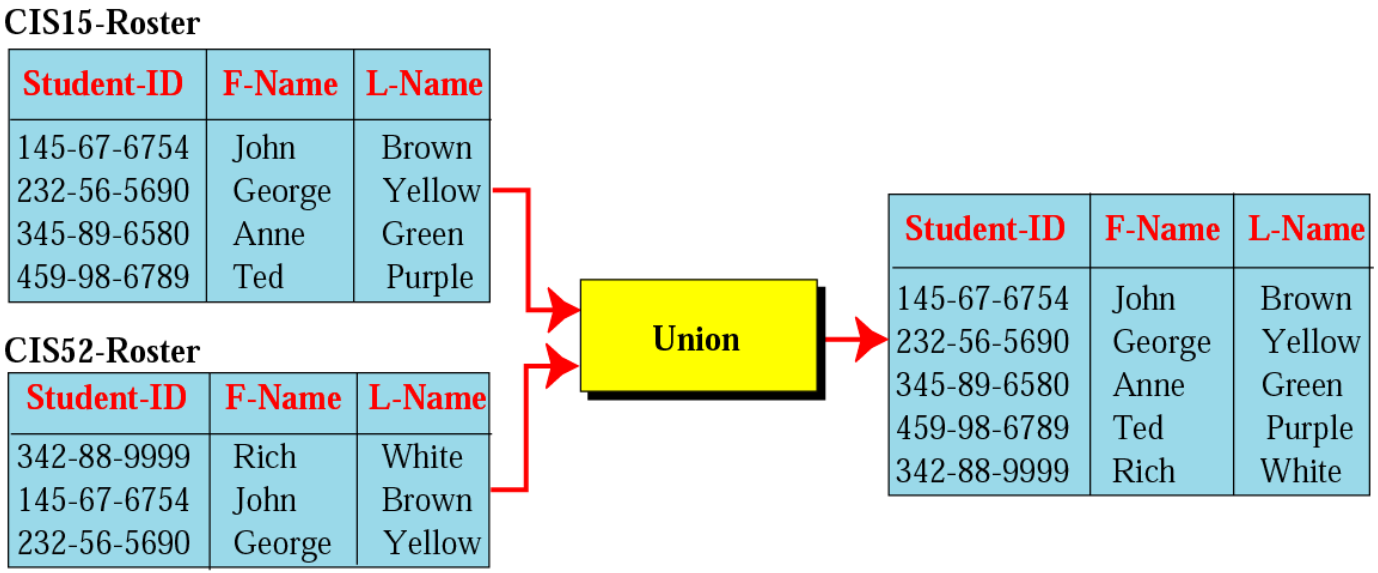
```
Q11:    SELECT    SALARY
        FROM      EMPLOYEE

Q11A:   SELECT    DISTINCT SALARY
        FROM      EMPLOYEE
```

SET OPERATIONS

- SQL has directly incorporated some set operations
  - There is a union operation (UNION),
  - and in *some versions* of SQL (MINUS) and intersection (INTERSECT)
- The resulting relations of these set operations are sets of tuples;
  - *duplicate tuples are eliminated from the result*
- The set operations apply only to *union compatible relations*;
  - the two relations must have the same attributes and the attributes must appear in the same order

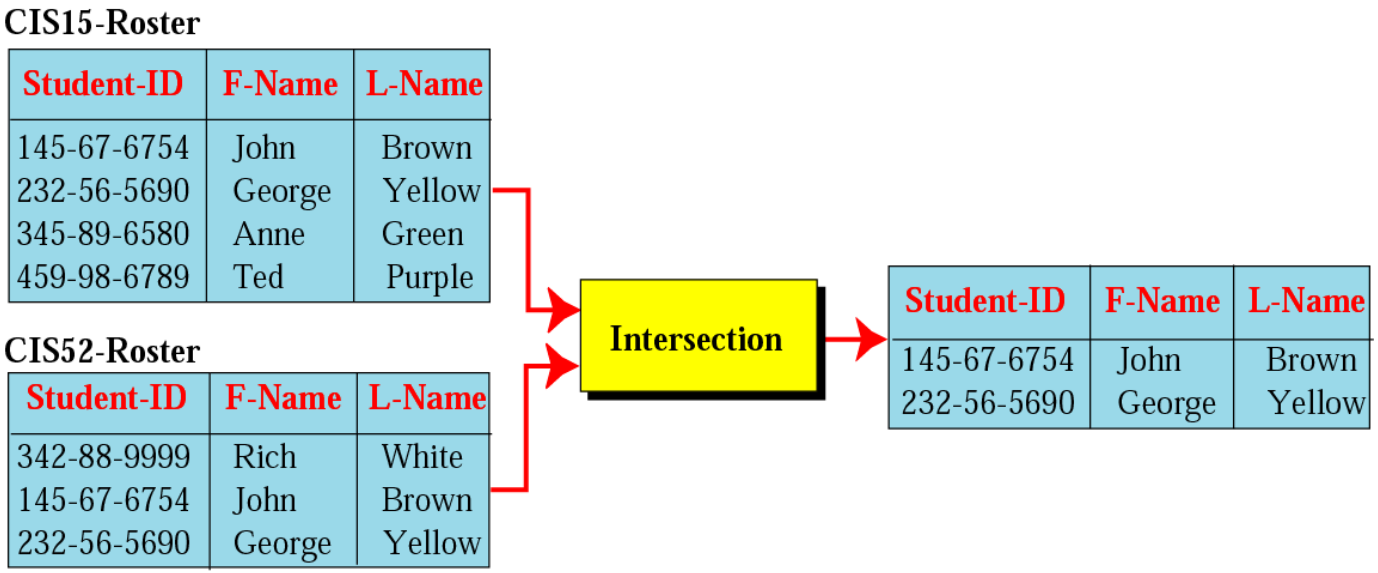
Union operation

- A binary operation.
- Creates a new relation in which each tuple is either in the first relation, in the second, or in both.
- The two relations must have the same attributes.



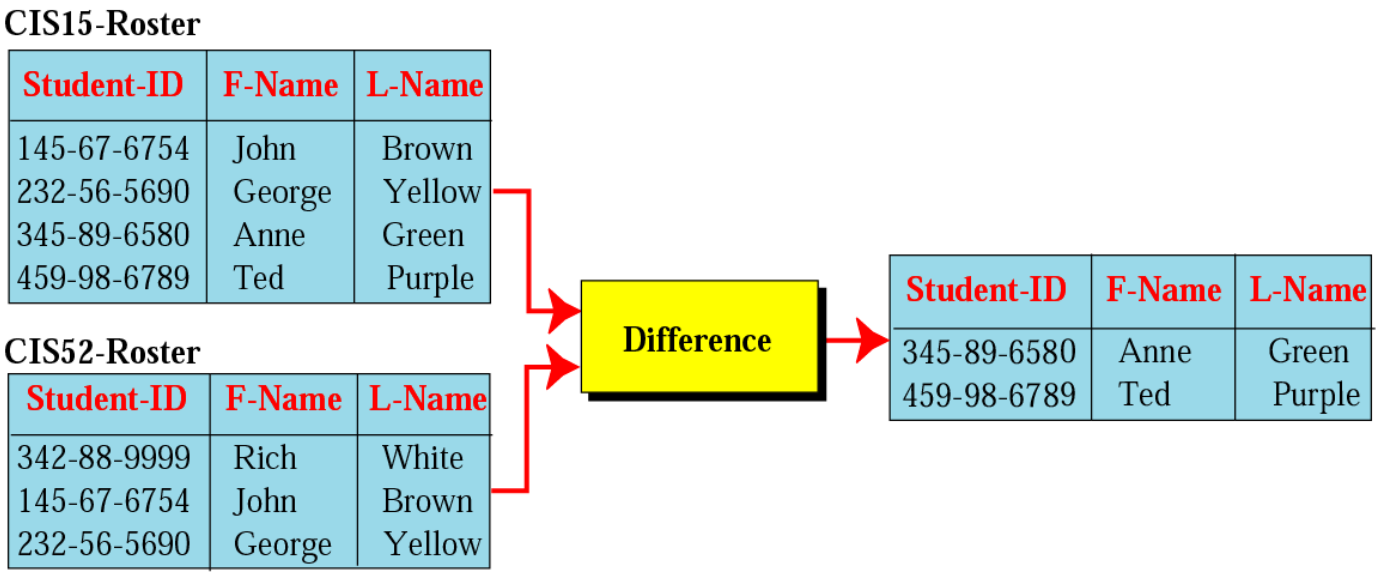
Intersection operation

- A binary operation.
- Creates a new relation in which each tuple is a member in both relations.
- The two relations must have the same attributes.



Difference operation

- A binary operation.
- Creates a new relation in which each tuple is in the first relation but not the second.
- The two relations must have the same attributes.



SET OPERATIONS (contd.)

- **Query 4**: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith' as a worker or as a manager of the department that controls the project.

```
Q4:   (SELECT   PNAME
       FROM     PROJECT, DEPARTMENT,
                EMPLOYEE
       WHERE    DNUM=DNUMBER AND
                MGRSSN=SSN AND LNAME='Smith')
       UNION
       (SELECT  PNAME
       FROM     PROJECT, WORKS_ON, EMPLOYEE
       WHERE    PNUMBER=PNO AND
                ESSN=SSN AND LNAME='Smith')
```